WEBSITE SECURITY ASSESSMENT & ISSUES REPORT

# HuntInKE

## Finding, documenting and fixing the issues in our own rental-housing platform

Every vulnerability discovered by manual code review and a 57-check adversarial suite is documented here — and fixed in the live prototype. Attacks include SQL injection, stored XSS, IDOR, brute force and upload abuse.

**Assignment:** find a website, document its issues, build a fixed prototype

**Prototype:** homelinkke.pages.dev

**Source:** github.com/Grimzeero78/HomeLinkKE

**Date:** 10 November 2026

57 CHECKS PASSED · 0 OPEN CRITICALS

# Table of Contents

**1. Introduction and Assignment Brief** **2**

**2. The Website Under Test** **2**

**3. Assessment Methodology** **3**

3.1 Manual code review . . . . . 3

3.2 Automated adversarial suite . . . . . 3

3.3 Browser-level verification . . . . . 4

**4. Security Vulnerabilities Found and Fixed** **4**

**5. Functional Bugs Found and Fixed** **5**

**6. Residual Risks, Documented and Accepted** **6**

**7. Automated Attack Suite Results** **7**

**8. Reproducing the Audit, and Deliverables** **8**

# 1. Introduction and Assignment Brief

The assignment asked for four things: find a page or website, find issues or vulnerabilities on it, document them, and create a prototype of the website that solves those issues. For this task we chose our own project, HuntInKE, a Kenya-wide rental-housing marketplace that connects renters with landlords through listings, search, a live map and in-app chat. Choosing our own site was deliberate: it gave us full access to the source code, so every finding could be traced to an exact line and every fix could be verified rather than guessed.

HuntInKE is a real, deployed product, not a mock-up. It runs on Cloudflare Pages with a Cloudflare D1 database, serves two domains, and is used by test landlords and renters across several Kenyan towns. That reality made the exercise meaningful: the issues we hunted for were issues that could genuinely expose real users, real listing data and real M-Pesa conversations. The prototype that solves the documented issues is the current HuntInKE build itself, published at homelinkke.pages.dev with its full source on GitHub.

This report is organised as follows. Chapter 2 describes the website under test. Chapter 3 explains the methodology used to hunt for issues. Chapters 4 and 5 document the security vulnerabilities and functional bugs that were found, together with the exact fixes applied in the prototype. Chapter 6 lists the residual risks we consciously accepted and why. Chapter 7 presents the results of a 57-check automated attack suite that proves the fixes work, and Chapter 8 explains how the lecturer or any reviewer can reproduce the entire audit from the repository in under five minutes.

# 2. The Website Under Test

HuntInKE (homelinkke.pages.dev) is a mobile-first marketplace for rental housing in Kenya. Renters browse listings by town, search by keyword, inspect photo galleries and key facts, contact landlords through in-app chat or WhatsApp, and save favourites. Landlords register, optionally verify their identity with documents, post listings with three to eight photos, and keep the vacancy count of each property up to date. A paid boost (KES 200 via M-Pesa) pins a listing to the top of its own town for seven days, and an admin panel handles verifications, boost payments, reports and moderation.

Technically the site is a single-page application backed by a serverless API. The frontend is vanilla JavaScript organised into twelve view modules; the backend is one Cloudflare Pages Function acting as a catch-all router over roughly 1,100 lines of code, with a D1 SQLite database behind it. Authentication is token based: a 40-character random session token stored client-side and sent as a Bearer header, with passwords hashed using PBKDF2-SHA-256 at 100,000 iterations and a per-user salt. Because the platform is young and growing quickly, it had never received a systematic security review before this assignment, which made it an honest target.

Three facts about the stack shaped the assessment. First, all requests flow through one API entry point, so ownership checks and rate limits concentrate in a small, auditable surface. Second, every database query is a prepared statement, which is the strongest possible starting position against SQL injection. Third, the client renders almost all content dynamically, so output escaping in the view modules is the single point of failure for cross-site scripting. The methodology in the next chapter was designed around these three facts.

# 3. Assessment Methodology

The assessment combined two complementary techniques, because neither alone tells the whole story. Manual code review reads the system the way an attacker would: following data from every user-controlled input (search terms, listing fields, photos, chat messages, profile fields) through storage, retrieval and final rendering, checking for injection sinks, missing escapes, missing ownership checks and abuse paths. Automated testing then fires real attack payloads at a running instance and records what actually happens, which catches the mistakes review misses.

## 3.1 Manual code review

We reviewed every route of the API and every render path of the client with an attacker's checklist: Are all SQL values parameterised? Is every user-controlled field escaped at render time? Does every write operation verify ownership on the server, not just in the interface? Are sessions invalidated correctly? Are privileged endpoints gated by role checks on the server? Are uploads strictly validated? Can any limit be flushed or bypassed? Each answer pointed either to a documented finding or to a control that the attack suite then tried to defeat.

## 3.2 Automated adversarial suite

The committed suite, tests/security_tests.py, runs about 57 adversarial checks against a live local instance and exits non-zero if any check fails. It registers its own accounts, posts its own listings with generated images, opens its own conversations, and then attacks: SQL injection payloads in search and login, stored and reflected XSS payloads, forged and empty authentication tokens, cross-user writes on listings and conversations, brute-force floods against login, registration, hint enumeration and messaging, boundary-value inputs, and a malicious SVG upload. Finally it verifies the security headers on every response.

### 3.3 Browser-level verification

Some properties cannot be proven from HTTP responses alone. The clearest example is stored XSS: the API deliberately stores a payload such as an onerror image tag as inert JSON data, and the real question is what happens when a browser renders it. We therefore opened the payload listing in a headless browser and counted injected elements: zero, with the payload visible only as plain text. The same browser runs confirmed the functional fixes described in chapter 5, including the ownership banner and the search behaviour.

# 4. Security Vulnerabilities Found and Fixed

Ten security issues survived review and testing. Each one is documented below with its severity, the weakness as found, and the concrete fix that now ships in the prototype. None of them remains open: the attack suite in chapter 7 re-tests every one of them on every run and currently reports 57 passes out of 57 checks.

| # | Severity | Vulnerability found | Fix in the prototype |
|---|---|---|---|
| S1 | High | GET /api/me answered 200 for forged or expired session tokens and for suspended accounts, leaking profile data to dead sessions. | Every request validates the token against the sessions table; 401 without a valid session, 403 for suspended accounts. |
| S2 | Medium | Stored XSS risk: user-controlled listing fields are rendered by 12 client view modules; a single unescaped interpolation would execute attacker HTML in every visitor's browser. | Every render path escapes via esc(); the period field passes through a literal-only mapper; payloads stored as inert JSON with nosniff; CSP blocks inline handlers. Proven in a real browser. |
| S3 | Medium | SVG upload slipped through the old data:image/ prefix check; SVG is script-capable outside an img tag. | Strict raster-only allow-list (png, jpeg, webp, gif), 1.2 MB per photo, 3 to 8 photos enforced. |
| S4 | Medium | The in-memory rate-limit map could be flushed by flooding it with unique keys, resetting brute-force counters mid-attack. | Sliding-window map prunes only expired entries, with an absolute memory cap. |
| S5 | Low | No rate limit on the one-time admin claim (setup) and on boost requests. | 5/hour/IP on setup plus a race-proof unique-row lock after the first claim; 5/hour/user on boost. |
| S6 | Low | Unbounded request bodies allowed arbitrarily large JSON uploads to the worker. | All bodies above 16 MB rejected with 413; the heaviest legitimate request is about 11 MB. |

| # | Severity | Vulnerability found | Fix in the prototype |
|---|---|---|---|
| S7 | Low | Early prototypes hashed passwords with a fast digest. | PBKDF2-SHA-256, 100,000 iterations, per-user 128-bit salt, constant-time comparison, and a generic wrong-username-or-password message that blocks user enumeration. |
| S8 | Low | API responses did not all carry an explicit Content-Type, inviting MIME sniffing. | One central JSON helper sets application/json with charset, no-store and nosniff on every API response. |
| S9 | Medium | IDOR surface: listings and conversations are addressed by sequential IDs, so any missing ownership check would expose another user's data. | Server-side ownership checks on every write (edit, delete, status, boost); conversations readable and writable only by their two participants; role gates on landlord and admin endpoints. |
| S10 | Low | Missing hardening headers would allow clickjacking, sniffing and referrer leakage. | CSP (self-only scripts), X-Frame-Options DENY with frame-ancestors none, nosniff, Referrer-Policy strict-origin-when-cross-origin, Permissions-Policy locking camera and microphone. |

Table 1. Security vulnerabilities documented in the audit and their fixes.

Two of these deserve emphasis because they are the classic web pitfalls this assignment was designed to surface. The stored-XSS defence (S2) is layered rather than single: payloads are stored as data, served only as JSON with nosniff, escaped at every render sink, and fenced by a Content-Security-Policy that blocks inline script execution even if an escape were ever forgotten. The IDOR defence (S9) follows one rule consistently: the server, never the client, decides who owns what, and every mutating route re-checks ownership against the session before touching a row.

# 5. Functional Bugs Found and Fixed

Security testing of a live site also surfaces functional bugs, because abuse paths and normal paths share the same code. Four significant ones were found while testing HuntInKE with real data and real accounts. They are documented here because a prototype that fixes vulnerabilities but stays broken for users would fail the spirit of the assignment: the goal is a website that is both safe and working.

| # | Severity | Bug found | Fix in the prototype |
|---|---|---|---|
| F1 | High (trust) | The listing-detail API reported isOwner true for admins as well as real owners, so the orange "This is your listing. Boost it for KES 200" banner rendered on every listing for the admin account, and owner-only UI leaked to non-owners. | isOwner is now strictly the real owner; admins keep moderation powers through a separate internal flag and see the same renter view as everyone else. |
| F2 | High (core UX) | Search returned zero results for real towns: the search view silently re-applied the renter's saved town as an exact filter, so searching "chuka" while the phone had saved "Nairobi" produced an impossible query. | A typed query never inherits the saved town; typing an exact town name now becomes the location filter. Verified live: saved town Nairobi plus search "chuka" finds the Chuka house. |
| F3 | Medium | Photos were never uploaded: the publish handler cleared the file input before reading it, but a FileList is a live object, so the upload list was always empty. | Snapshot the files before clearing the input; camera capture input fixed for Android in-app browsers. |
| F4 | Low | Several endpoints in early builds had no abuse limits at all. | Full sliding-window limits: login 15/min/IP and 8/min/username, registration 8/h/IP, hint 10/min/IP, messages 30/min, conversations 10/min, reads 240/min, view counter deduplicated per IP per listing per 6 hours. |

Table 2. Functional and authorisation bugs found while testing the live site.

Bug F1 is a good example of how a small convenience in code becomes a trust problem in product. The developer had written the owner check as "is the session user the owner, or an admin", which reads naturally but hands every admin a page that lies to them: it says the property is theirs and invites them to pay for it. The fix separates two questions that the old code merged - who may moderate, and who owns - and the suite now asserts that only the real owner sees the banner. Bug F2 similarly shows how a personalisation feature (remembering your town) can silently corrupt search until the product looks empty; the fix makes typed intent always win over stored preferences.

# 6. Residual Risks, Documented and Accepted

A serious assessment does not end with an empty risk register; it ends with an honest one. Four residual risks were identified, assessed, and consciously accepted for the current scale of the platform, each with its mitigation and a recorded next step. Publishing them keeps the prototype honest about what it does and does not promise.

| # | Risk | Why accepted | Mitigations and next step |
|---|---|---|---|
| R1 | /api/hint/:username reveals whether a username exists and returns the user's password hint; no e-mail service exists for real account recovery. | Deliberate trade-off for a market with low e-mail access; hints are forced to differ from the password at registration. | Rate limited to 10/min/IP and hints capped at 120 characters. Next step: SMS one-time-code self-service reset. |
| R2 | Rate limiting is per-isolate and held in memory. | Cloudflare's edge absorbs distributed floods; the in-memory layer is defence in depth. | Prune-safe windows plus edge protection. Next step: durable counters if abuse is observed. |
| R3 | Photos are stored as data URLs inside the D1 database. | Strict MIME and size caps make this safe at current scale. | Allow-listed formats, 1.2 MB per photo, 16 MB body cap. Next step: an R2 object bucket at scale. |
| R4 | The platform never handles rent or deposits. | By design: every surface repeats the view-first, pay-later rule, so there is no payment data to steal. | M-Pesa happens off-platform, face-to-face, between tenant and landlord. |

Table 3. Residual risks with their accepted rationale and mitigations.

# 7. Automated Attack Suite Results

The committed suite re-runs the entire assessment in about one minute and exits non-zero if a single check regresses, which makes it both evidence and a safety net for future changes. The latest full run, executed against a fresh local instance on 10 November 2026, reports 57 passes and 0 failures. The raw log ships with the repository as tests/evidence-latest.txt.

| 57 / 57 | 14 | 0 |
|---|---|---|
| checks passed | issues fixed | open criticals |

| Category | Checks | Result | Sample evidence from the run |
|---|---|---|---|
| A. Bootstrap | 6 | 6 PASS | Three accounts and two listings created for multi-user tests. |
| B. Security headers | 7 | 7 PASS | CSP, X-Frame-Options DENY, nosniff, Referrer-Policy, Permissions-Policy present on every response. |
| C. Auth and sessions | 9 | 9 PASS | No token 401; forged token 401; seeker and landlord blocked from admin API; token dead after logout. |
| D. SQL injection | 8 | 8 PASS | OR 1=1, UNION SELECT pass_hash and DROP TABLE payloads all parameterised; listings table alive; no credential columns leaked. |

| Category | Checks | Result | Sample evidence from the run |
|---|---|---|---|
| E. XSS | 4 | 4 PASS | onerror image payload stored as inert JSON data with nosniff; browser render shows 0 injected elements. |
| F. IDOR | 7 | 7 PASS | Edit, delete, boost and status of another landlord's listing blocked; stranger cannot open or post in a conversation. |
| G. Privacy | 2 | 2 PASS | Anonymous listing detail returns empty owner phone fields. |
| H. Input validation | 8 | 8 PASS | Zero and negative rent rejected; 10 KB title capped to 90 characters; 2 or 9 photos rejected; SVG upload rejected. |
| I. Rate limiting | 4 | 4 PASS | Login, hint, registration and message floods all hit 429. |
| Total | 57 | 57 PASS / 0 FAIL | Exit code 0; full log in tests/evidence-latest.txt. |

Table 4. Attack suite results by category (latest run: 10 November 2026).

Every category maps back to the findings of chapters 4 and 5: category D exercises the parameterisation claimed in S-section findings, categories F and C exercise the ownership and role gates, category I exercises the rate limits, and the browser check in category E closes the loop that HTTP-level checks cannot. Because the suite is committed alongside the code, any future change that weakens a control fails continuous checks instead of silently shipping.

# 8. Reproducing the Audit, and Deliverables

The entire assessment is reproducible from the public repository with two commands. Cloning the project, installing dependencies and starting a local instance takes about two minutes; the suite then runs in about one minute and prints each check with its evidence. This reproducibility is deliberate: a security claim that cannot be re-tested is only an opinion.

```
git clone https://github.com/Grimzeero78/HomeLinkKE && cd HomeLinkKE
npm install
npx wrangler pages dev public --d1 DB=homelinkke-local --port 8788
python3 tests/security_tests.py http://127.0.0.1:8788
```

The assignment produces four concrete deliverables, each committed and publicly reachable; they are summarised in Table 5 together with where to find them.

| Deliverable | What it is | Where to find it |
|---|---|---|
| Live prototype | The fixed website itself, with every documented issue solved. | homelinkke.pages.dev (mirror: homelinke.pages.dev) |

| Deliverable | What it is | Where to find it |
|---|---|---|
| Source repository | Complete source code, this report, the test suite and the audit trail. | github.com/Grimzeero78/HomeLinkKE |
| Evidence log | Raw output of the latest full suite run: 57 passes, 0 failures. | tests/evidence-latest.txt (in the repository) |
| Attack suite | The re-runnable 57-check adversarial suite (standard library only). | tests/security_tests.py (in the repository) |

Table 5. Assignment deliverables and their locations.

In summary: the exercise found fourteen genuine issues across security and functionality, ranging from a forged-token acceptance and an IDOR-prone ownership model to a search bug that hid half the product from its own users. All fourteen are fixed in the prototype, the remaining risks are documented with their rationale, and a permanent, committed test suite now guards every claim this report makes.